

Lezione 4: Logica classica e quantificatori

Stefano M. Nicoletti

1 Lezione 4: Logica classica e quantificatori

1.1 Scopo della lezione

Nella Lezione 3 abbiamo collegato le tattiche di Lean alle regole di introduzione e di eliminazione della deduzione naturale. In questa lezione estendiamo quel quadro in due direzioni. Prima introduciamo la riduzione all'assurdo come regola classica. Poi passiamo dalla logica proposizionale alla logica dei predicati, imparando a rappresentare oggetti, proprietà, funzioni, relazioni e quantificatori.

Il percorso della lezione è il seguente:

- distinguere le regole costruttive dai principi classici;
- applicare la riduzione all'assurdo;
- costruire un semplice linguaggio del primo ordine in Lean;
- formalizzare enunciati contenenti $\forall$ ed $\exists$;
- riconoscere e applicare le regole $\forall I$, $\forall E$, $\exists I$ ed $\exists E$.

1.2 Dalle regole costruttive alla logica classica

Le regole viste finora hanno una lettura costruttiva diretta: ogni dimostrazione specifica come costruire il termine richiesto. Per dimostrare una congiunzione costruiamo entrambe le componenti; per dimostrare un'implicazione costruiamo una funzione.

Nella logica costruttiva di base di Lean, il terzo escluso $A \vee \neg A$ e l'eliminazione della doppia negazione $\neg\neg A \rightarrow A$ non sono disponibili automaticamente. Lean permette però di usare principi classici in modo locale ed esplicito (Fonte: [Theorem Proving in Lean 4, Classical Logic](#)).

Questa distinzione riguarda gli strumenti ammessi nella dimostrazione. Non significa che Lean vieti la logica classica ma il ricorso a un principio classico deve rimanere esplicitamente visibile nel termine costruito.

1.3 Riduzione all'assurdo

La riduzione all'assurdo classica permette di dimostrare A mostrando che l'assunzione $\neg A$ conduce a `False`. In Lean la applichiamo con `Classical.byContradiction`.

$$\frac{\frac{\vdots}{\perp} 1}{A} \text{ RAA}$$

Riduzione all'assurdo, RAA

Lo schema operativo è il seguente:

1. applichiamo la riduzione all'assurdo; 2. introduciamo l'assunzione $\neg A$; 3. usiamo le assunzioni disponibili per ottenere False; 4. usiamo esattamente questa contraddizione.

Vogliamo dimostrare che piove, date due assunzioni: se non piove, non prendo l'ombrello; prendo l'ombrello. Dopo aver applicato la riduzione all'assurdo, il nuovo compito è mostrare che supporre che non piova produce una contraddizione.

```

theorem lecture04_reductio_ad_absurdum
  (Piove PrendoOmbrello : Prop)
  (hNonPioveNonOmbrello : ¬Piove → ¬PrendoOmbrello)
  (hPrendoOmbrello : PrendoOmbrello) :
  Piove := by
    apply Classical.byContradiction
    intro hNonPiove
    have hNonPrendoOmbrello :=
      hNonPioveNonOmbrello hNonPiove
    have hContraddizione :=
      hNonPrendoOmbrello hPrendoOmbrello
    exact hContraddizione

```

La prima assunzione trasforma $\neg \text{Piove}$ in $\neg \text{PrendoOmbrello}$. Quest'ultimo fatto è una funzione da PrendoOmbrello a False; applicandola all'assunzione hPrendoOmbrello , otteniamo la contraddizione richiesta.

1.4 Un limite della logica proposizionale

Nella logica proposizionale trattiamo ogni enunciato come una proposizione completa. Potremmo usare:

- P per «Ipazia è una filosofa»;
- Q per «Hannah Arendt è una filosofa»;
- R per «Ipazia è curiosa».

Questa rappresentazione nasconde però la struttura interna degli enunciati. Non mostra che P e Q attribuiscono la stessa proprietà a due oggetti diversi. Non permette nemmeno di esprimere direttamente «ogni filosofa è curiosa» oppure «esiste una filosofa curiosa».

La logica dei predicati rende visibili gli oggetti di cui parliamo e le proprietà o relazioni che attribuiamo loro.

1.5 Il dominio del discorso

Ogni formalizzazione parte da un *dominio*: la collezione degli oggetti che stiamo considerando. In questa lezione usiamo un solo tipo generale:

```
variable (Cosa : Type)
```

Cosa rappresenta il dominio del discorso. Un termine $x : \text{Cosa}$ è un elemento del dominio. La scelta di un unico dominio corrisponde alla presentazione a un solo dominio tipica della logica del primo ordine (Fonte: [Logic and Proof, First Order Logic in Lean](#)).

Possiamo poi introdurre termini che interpretiamo come oggetti nominati:

```
variable (Cosa : Type)
variable (ipazia hannah roma atene : Cosa)
```

Queste dichiarazioni non affermano ancora alcuna proposizione. Stabiliscono soltanto che ipazia, hannah, roma e atene sono termini del dominio.

1.6 Predicati: proprietà degli oggetti

Un predicato con un argomento prende un oggetto del dominio e produce una proposizione. Per esempio:

```
variable (Cosa : Type)
variable (Persona Citta Filosofa Curiosa : Cosa → Prop)
```

Filosofa : Cosa → Prop è una proprietà che possiamo applicare a oggetti diversi:

```
variable (Cosa : Type)
variable (ipazia hannah : Cosa)
variable (Filosofa : Cosa → Prop)

#check Filosofa ipazia -- Filosofa ipazia : Prop
#check Filosofa hannah -- Filosofa hannah : Prop
```

Le espressioni `Filosofa ipazia` e `Filosofa hannah` sono proposizioni distinte, ma condividono lo stesso predicato. La struttura comune non viene più nascosta dietro lettere proposizionali indipendenti.

Linguaggio naturale	Lean
Ipazia è curiosa	Curiosa ipazia
Hannah Arendt è curiosa	Curiosa hannah
Io è una luna di Giove	LunaDiGiove io
Roma è piovosa	Piove roma

1.7 Funzioni: costruire nuovi termini

Una funzione prende uno o più oggetti del dominio e restituisce un oggetto del dominio. Per esempio:

```
variable (Cosa : Type)
variable (madreDi : Cosa → Cosa)
```

`madreDi ipazia : Cosa` è un nuovo termine, che interpretiamo come «la madre di Ipazia». Poiché il risultato ha ancora tipo `Cosa`, possiamo usarlo come argomento di un predicato:

```
variable (Cosa : Type)
variable (ipazia : Cosa)
variable (madreDi : Cosa → Cosa)
variable (Curiosa : Cosa → Prop)

#check madreDi ipazia
#check Curiosa (madreDi ipazia)
```

Le funzioni permettono di costruire termini progressivamente più complessi:

Termine	Lettura
<code>ipazia</code>	Ipazia
<code>madreDi ipazia</code>	la madre di Ipazia
<code>madreDi (madreDi ipazia)</code>	la madre della madre di Ipazia

Nella logica del primo ordine una funzione è totale e restituisce un solo risultato per ogni argomento. Una funzione restituisce un oggetto; un predicato restituisce una proposizione.

1.8 Relazioni tra oggetti

Un predicato con due o più argomenti esprime una relazione. In Lean possiamo dichiarare:

```
variable (Cosa : Type)
variable (Conosce Visita : Cosa → Cosa → Prop)
```

Otteniamo così proposizioni come:

```
variable (Cosa : Type)
variable (ipazia hannah atene : Cosa)
variable (Conosce Visita : Cosa → Cosa → Prop)

#check Conosce ipazia hannah
#check Visita ipazia atene
```

L'ordine degli argomenti è significativo. `Conosce ipazia hannah` rappresenta «Ipazia conosce Hannah Arendt»; `Conosce hannah ipazia` rappresenta invece «Hannah Arendt conosce Ipazia».

1.9 Parametri locali e assunzioni

La parola chiave `variable` introduce parametri locali. Dentro una `section` possiamo assumere temporaneamente un dominio e un vocabolario:

```
section LinguaggioDeiPredicati

variable (Cosa : Type)
variable (ipazia hannah : Cosa)
variable (Filosofa Curiosa : Cosa → Prop)
variable (madreDi : Cosa → Cosa)
variable (Conosce : Cosa → Cosa → Prop)

end LinguaggioDeiPredicati
```

La sezione delimita la portata di questi parametri. Inoltre Lean aggiunge a un teorema soltanto i parametri che compaiono effettivamente nel suo enunciato.

Dichiarare `Filosofa : Cosa → Prop` non significa assumere che qualcuno sia una filosofa. Per avere una dimostrazione che Ipazia è una filosofa serve una vera assunzione:

```
variable (Cosa : Type)
variable (ipazia : Cosa)
variable (Filosofa : Cosa → Prop)
variable (hIpaziaFilosofa : Filosofia ipazia)

#check hIpaziaFilosofa
```

Il vocabolario determina quali proposizioni possiamo formulare; le assunzioni forniscono dimostrazioni di alcune di quelle proposizioni.

1.10 Variabili e quantificatori

Una variabile come `x : Cosa` rappresenta un oggetto non ancora specificato del dominio. Possiamo costruire espressioni aperte come:

```
variable (Cosa : Type)
variable (x : Cosa)
variable (Persona Filosofia Curiosa : Cosa → Prop)

#check Filosofia x
#check Persona x ∧ Filosofia x
#check Persona x → Filosofia x → Curiosa x
```

Per ottenere enunciati che riguardano tutti gli oggetti oppure almeno un oggetto usiamo i quantificatori:

- $\forall$ si legge «per ogni»;
- $\exists$ si legge «esiste almeno un».

Linguaggio naturale	Lean
Ogni persona è curiosa	$\forall x : \text{Cosa}, \text{Persona } x \rightarrow \text{Curiosa } x$
Qualche persona è curiosa	$\exists x : \text{Cosa}, \text{Persona } x \wedge \text{Curiosa } x$
Ogni filosofa è curiosa	$\forall x : \text{Cosa}, \text{Filosofa } x \rightarrow \text{Curiosa } x$
Esiste una filosofa curiosa	$\exists x : \text{Cosa}, \text{Filosofa } x \wedge \text{Curiosa } x$

All'inizio scriviamo esplicitamente $x : \text{Cosa}$: il quantificatore introduce la variabile x e ne dichiara il dominio. Lean può talvolta ricavare il tipo dai predicati, ma la forma esplicita rende più chiara la struttura logica.

1.11 Implicazione e congiunzione nei quantificatori

Due schemi compaiono frequentemente nelle formalizzazioni:

```
variable (Cosa : Type)
variable (Filosofa Curiosa : Cosa → Prop)

#check ∀ x : Cosa, Filosofa x → Curiosa x
#check ∃ x : Cosa, Filosofa x ∧ Curiosa x
```

Nel primo caso prendiamo una cosa qualsiasi e diciamo: *se* è una filosofa, allora è curiosa. Il quantificatore universale copre l'intero dominio; l'implicazione restringe l'affermazione agli oggetti che soddisfano *Filosofa*.

Nel secondo caso cerchiamo un singolo testimone che sia *sia* una filosofa *sia* curiosa. Per questo le due proprietà sono collegate da una congiunzione.

Consideriamo altri esempi:

```
variable (Cosa : Type)
variable (Studentessa Legge Comprende : Cosa → Prop)

#check ∀ x : Cosa, Studentessa x → Legge x
#check ∃ x : Cosa, Studentessa x ∧ Legge x
#check ∀ x : Cosa, Studentessa x → Legge x → Comprende x
#check ∃ x : Cosa, Studentessa x ∧ Legge x ∧ Comprende x
```

`#check` verifica che ogni espressione sia ben formata e ne mostra il tipo. Non dimostra che l'enunciato sia vero.

1.12 Funzioni dentro le formule

I termini costruiti mediante funzioni possono comparire dentro predicati, relazioni e quantificatori:

```
variable (Cosa : Type)
variable (ipazia : Cosa)
variable (Studentessa Legge Curiosa : Cosa → Prop)
variable (madreDi : Cosa → Cosa)
variable (Conosce : Cosa → Cosa → Prop)
```

```
#check Curiosa (madreDi ipazia)

#check ∀ x : Cosa,
  Studentessa x → Conosce x (madreDi x)

#check ∃ x : Cosa,
  Studentessa x ∧ Legge (madreDi x)
```

Le parentesi rendono visibile l'ordine della costruzione: prima formiamo il termine `madreDi x`, poi lo usiamo come argomento del predicato `Legge` o della relazione `Conosce`.

1.13 Quantificatori annidati

Una formula può contenere più quantificatori. Consideriamo la relazione `Conosce : Cosa → Cosa → Prop`:

```
variable (Cosa : Type)
variable (Persona : Cosa → Prop)
variable (Conosce : Cosa → Cosa → Prop)

-- Ogni persona conosce qualcuno.
#check ∀ x : Cosa, Persona x →
  ∃ y : Cosa, Persona y ∧ Conosce x y

-- Qualcuno conosce ogni persona.
#check ∃ x : Cosa, Persona x ∧
  ∀ y : Cosa, Persona y → Conosce x y

-- Nessuno conosce ogni persona.
#check ∃¬ x : Cosa, Persona x ∧
  ∀ y : Cosa, Persona y → Conosce x y
```

Le prime due proposizioni non sono equivalenti. In «ogni persona conosce qualcuno», il testimone y può cambiare al cambiare di x . In «qualcuno conosce ogni persona», dobbiamo invece trovare un unico x che soddisfi la condizione per tutte le persone y .

1.14 Introduzione del quantificatore universale, '∀'

$$\frac{A(x)}{\forall y A(y)} \forall I$$

Introduzione del quantificatore universale, $\forall I$

Per dimostrare $\forall y : \text{Cosa}, A y$, introduciamo una cosa arbitraria x . Il goal diventa $A x$. Dimostriamo $A x$ senza usare proprietà particolari di x ; possiamo allora concludere che A vale per ogni oggetto del dominio.

Nel seguente esempio soltanto il primo `intro` applica $\forall I$. I due `intro` successivi introducono le assunzioni delle implicazioni.

```
theorem lecture04_forall_intro
  (Cosa : Type)
  (Persona Curiosa : Cosa → Prop) :
  ∀ y : Cosa, Persona y → Curiosa y → Curiosa y := by
  intro x
  intro hXPersona
  intro hXCuriosa
  exact hXCuriosa
```

Il primo passo introduce $x : \text{Cosa}$ come oggetto arbitrario. Il goal diventa $\text{Persona } x \rightarrow \text{Curiosa } x \rightarrow \text{Curiosa } x$. Introduciamo quindi le due assunzioni e usiamo esattamente `hXCuriosa`.

1.14.1 La condizione di arbitrarietà

L'oggetto introdotto da $\forall I$ non può essere una persona particolare scelta per una proprietà speciale. In Lean, `intro x` introduce un nuovo oggetto del tipo sul quale stiamo quantificando. Nel nostro esempio non sappiamo nulla di x prima di introdurre separatamente le assunzioni $\text{Persona } x$ e $\text{Curiosa } x$.

1.15 Eliminazione del quantificatore universale, ' $\forall E$ '

$$\frac{\forall x A(x)}{A(t)} \forall E$$

Eliminazione del quantificatore universale, $\forall E$

Se abbiamo $\forall x : \text{Cosa}, A x$, possiamo scegliere un termine t del dominio e applicare la dimostrazione universale a t . Otteniamo così $A t$.

Vogliamo dimostrare che Marta legge, data l'assunzione che ogni studentessa legge e che Marta è una studentessa.

```
theorem lecture04_forall_elim
  (Cosa : Type)
  (Studentessa Legge : Cosa → Prop)
  (marta : Cosa)
  (h0gniStudentessaLegge :
    ∀ x : Cosa, Studentessa x → Legge x)
  (hMartaStudentessa : Studentessa marta) :
  Legge marta := by
  have hSeMartaStudentessaAlloraLegge :=
    h0gniStudentessaLegge marta
  have hMartaLegge :=
    hSeMartaStudentessaAlloraLegge hMartaStudentessa
  exact hMartaLegge
```


Applicando l'assunzione universale a marta otteniamo Studentessa marta $\rightarrow$ Legge marta. Appliciamo poi questa implicazione all'assunzione hMartaStudentessa.

1.16 Introduzione del quantificatore esistenziale, 'EI'

$$\frac{A(t)}{\exists x A(x)} \text{EI}$$

Introduzione del quantificatore esistenziale, EI

Per dimostrare $\exists x : \text{Cosa}, A x$ dobbiamo scegliere un testimone t e dimostrare $A t$. In Lean usiamo il costruttore `Exists.intro`.

```
theorem lecture04_exists_intro
  (Cosa : Type)
  (Legge : Cosa → Prop)
  (marta : Cosa)
  (hMartaLegge : Legge marta) :
  ∃ x : Cosa, Legge x := by
  apply Exists.intro marta
  exact hMartaLegge
```

Scegliendo marta come testimone, il goal esistenziale diventa Legge marta. Questa proposizione è già disponibile come assunzione.

1.17 Eliminazione del quantificatore esistenziale, 'EE'

$$\frac{\exists x A(x) \quad \begin{array}{c} \overline{A(y)}^1 \quad 1 \\ \vdots \\ B \end{array}}{B} \text{EE}$$

Eliminazione del quantificatore esistenziale, EE

Se abbiamo $\exists x : \text{Cosa}, A x$, sappiamo che esiste un testimone, ma non possiamo presupporre quale sia. Per ottenere una conclusione B , introduciamo un testimone arbitrario y e l'assunzione $A y$; da questa assunzione dimostriamo B . La conclusione non deve dipendere dall'identità del testimone.

Qualcuno ha fatto una segnalazione. Inoltre, se qualcuno fa una segnalazione, inizia un'indagine. Vogliamo concludere che inizia un'indagine.

```
theorem lecture04_exists_elim
  (Cosa : Type)
  (HaFattoSegnalazione : Cosa → Prop)
  (IniziaIndagine : Prop)
  (hQualcunoHaFattoSegnalazione :
```

```

    ∃ x : Cosa, HaFattoSegnalazione x)
(hChiFaSegnalazioneAvviaIndagine :
  ∀ x : Cosa, HaFattoSegnalazione x → IniziaIndagine) :
  IniziaIndagine := by
apply Exists.elim hQualcunoHaFattoSegnalazione
intro y
intro hYHaFattoSegnalazione
have hIndagine :=
  hChiFaSegnalazioneAvviaIndagine y hYHaFattoSegnalazione
exact hIndagine

```

`Exists.elim` apre l'esistenziale. Introduciamo il testimone `y` e il fatto che `y` abbia fatto una segnalazione. Applichiamo quindi l'assunzione universale a `y` e al fatto disponibile. La conclusione `IniziaIndagine` non contiene `y` e non dipende dalla sua identità.

1.18 Combinare le regole

Le regole dei quantificatori vengono spesso usate insieme. Supponiamo che qualcuno abbia un ombrello e che chiunque abbia un ombrello resti asciutto. Vogliamo dimostrare che qualcuno resta asciutto.

```

theorem lecture04_exists_elim_combined
  (Cosa : Type)
  (HaOmbrello RestaAsciutta : Cosa → Prop)
  (hQualcunaHaOmbrello : ∃ x : Cosa, HaOmbrello x)
  (hChiHaOmbrelloRestaAsciutta :
    ∀ x : Cosa, HaOmbrello x → RestaAsciutta x) :
  ∃ x : Cosa, RestaAsciutta x := by
apply Exists.elim hQualcunaHaOmbrello
intro x
intro hXHaOmbrello
apply Exists.intro x
have hXRestaAsciutta :=
  hChiHaOmbrelloRestaAsciutta x hXHaOmbrello
exact hXRestaAsciutta

```

La dimostrazione combina tre passaggi:

1. `Exists.elim` elimina il primo esistenziale e rende disponibile un testimone `x` con `HaOmbrello x`; 2. l'applicazione di `hChiHaOmbrelloRestaAsciutta` a `x` usa $\forall E$; 3. `Exists.intro x` usa lo stesso oggetto come testimone del nuovo esistenziale.

1.19 Strategia operativa

Quando incontriamo una formula quantificata, possiamo porci alcune domande.

- Il goal comincia con $\forall$? Introduciamo un oggetto arbitrario con `intro`.
- Abbiamo un'assunzione universale? Appliciamola a un termine del dominio.
- Il goal comincia con $\exists$? Scegliamo un testimone con `Exists.intro`.

- Abbiamo un'assunzione esistenziale? Apriamola con `Exists.elim`, senza presupporre l'identità del testimone.
- Sono presenti più quantificatori? Controlliamo attentamente il loro ordine e quali testimoni possono dipendere da quali variabili.

Queste domande estendono la strategia usata per i connettivi proposizionali: guardiamo il costruttore principale del goal per scegliere una regola di introduzione e guardiamo le assunzioni composte per scegliere una regola di eliminazione.

1.20 Fonti e approfondimenti

- Jeremy Avigad, Robert Y. Lewis e Floris van Doorn, *Logic and Proof*, «Natural Deduction Rules»: https://leanprover.github.io/logic_and_proof_lean3/nd_quickref.html.
- Lean developers, *The Lean Language Reference*, «Quantifiers»: <https://lean-lang.org/doc/reference/latest/Basic-Propositions/Quantifiers/>.
- Jeremy Avigad, Robert Y. Lewis e Floris van Doorn, *Logic and Proof*, «First Order Logic in Lean»: https://leanprover.github.io/logic_and_proof_lean3/first_order_logic_in_lean.html.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong e Sebastian Ullrich, *Theorem Proving in Lean 4*, «Classical Logic»: https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/.